

Recuperatorio del Parcial 1 — 2026

Sistemas Operativos / Sistemas Operativos y Redes

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2026

Resumen

Resolución completa del Recuperatorio del Parcial 1 del año 2026 de la cátedra de Sistemas Operativos (Lic. en Ciencias de la Computación, UNRC). El parcial consta de cinco ejercicios que abarcan: (i) verdadero/falso sobre llamadas al sistema básicas (`exec`, `exit`, `fork`, `wait`, `printf`, `traps`); (ii) conteo de procesos creados por una cadena de `fork()` con un `exec()` final; (iii) análisis de un programa de comunicación padre/hijo con `pipe()` y propuesta de corrección; (iv) análisis de un escenario clásico de *deadlock* con dos mutex tomados en orden distinto entre tareas; (v) planificación *Round Robin* con `quantum = 2` sobre tres procesos. Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

Índice

1. Ejercicio 1 — V/F sobre syscalls (2 pts)	1
2. Ejercicio 2 — Cuántos procesos crea esta cadena de forks (2 pts)	4
3. Ejercicio 3 — Comunicación con <code>pipe()</code> : ¿correcto? (2 pts)	5
4. Ejercicio 4 — Transferencias bancarias y <i>deadlock</i> (2 pts)	7
5. Ejercicio 5 — Round Robin con <code>quantum = 2</code> (2 pts)	9
Resumen del recuperatorio	11

1. Ejercicio 1 — V/F sobre syscalls (2 pts)

Enunciado. Determinar la verdad o falsedad de las siguientes sentencias. Justificar las falsas en forma breve, completa y con claridad.

1.A — `exec(path, args)`

“La llamada al sistema `exec(path, args)` crea un nuevo proceso y carga las secciones de código y datos del programa y comienza su ejecución.”

Respuesta

F. `exec` no crea un nuevo proceso. Lo que hace es *reemplazar* la imagen del proceso *actual*: mismo PID, mismo descriptor en el kernel, pero todo el segmento de código, datos y stack se reescribe con los del nuevo programa. La ejecución continúa en el `main` del nuevo programa.

Para *crear* un nuevo proceso se usa `fork()`. La combinación clásica es `fork` + `exec`: el padre hace `fork()` (creando un proceso hijo idéntico) y el hijo hace `exec()` para correr otro programa. Así es como `bash`, por ejemplo, lanza cualquier comando.

1.B — Inicio de la ejecución

“Un proceso siempre comienza su ejecución por la función `main()`.”

Respuesta

F. Antes de `main()` se ejecuta el *startup code* del runtime de C (CRT). El punto de entrada real del binario es `_start` (suministrado por la `libc`), que se encarga de:

- Preparar `argc` y `argv` desde la pila inicial.
- Inicializar las variables globales en `.data` y poner en cero la sección `.bss`.
- Llamar a los constructores estáticos (en C++) y a las funciones registradas con atributo `constructor`.
- Recién entonces invocar a `main()`.
- Al volver de `main`, llamar a las funciones de `atexit` y a `_exit` con el valor de retorno.

Para el programador C, `main` parece ser el primer código que se ejecuta, pero el binario en sí arranca varios pasos antes.

1.C — `printf` como syscall

“La función `printf(fmt, ...)` es una llamada al sistema.”

Respuesta

F. `printf` es una función de la **biblioteca estándar de C** (`libc`), *no* una syscall. Internamente:

1. Formatea los argumentos según la cadena `fmt` y los coloca en un buffer interno (línea o bloque, según el tipo de stream).
2. Eventualmente llama a la syscall real `write(STDOUT_FILENO, buffer, n)` para volcar los bytes al kernel.

Por eso es posible reemplazar `printf` con otra implementación (e.g. versiones `nano-printf` en sistemas embebidos) sin cambiar el kernel: la única syscall involucrada es `write`.

1.D — `exit(n)` y el *zombie*

“La llamada al sistema `exit(n)` finaliza el proceso y en el descriptor del proceso queda el `exit code = n` hasta que el proceso padre ejecute `wait(&c)`.”

Respuesta

V. Cuando un proceso ejecuta `exit(n)`:

- El kernel libera los recursos del proceso (memoria, descriptors abiertos, etc.).
- **No** libera el descriptor (`task_struct` en Linux) inmediatamente: lo deja en estado ZOMBIE guardando los 8 bits inferiores de `n` en `exit_code`.
- Le envía la señal `SIGCHLD` al padre.
- Cuando el padre invoca `wait(&c)` (o `waitpid`), el kernel copia el exit code en `c`, libera el descriptor del zombie y este desaparece definitivamente.

Si el padre nunca llama a `wait`, el zombie queda indefinidamente en la tabla de procesos hasta que el padre muera y `init/systemd` adopte al huérfano y lo coseche.

1.E — `fork()`

“La llamada al sistema `fork()` copia la imagen (código, datos y *stack*) en la memoria del proceso invocante previa creación de un nuevo descriptor en la tabla de procesos en el kernel con un nuevo *PID*.”

Respuesta

F. Hay dos problemas con la afirmación:

1. “**en la memoria del proceso invocante**” está confuso: la imagen se copia (lógicamente) en la memoria *del nuevo proceso* (el hijo), no en la del invocante (que sería el padre, donde ya está la imagen original). El hijo recibe su propio espacio de direcciones.
2. En la práctica, los SO modernos **no copian la memoria entera** al hacer `fork`: usan **copy-on-write (COW)**. El padre y el hijo comparten las mismas páginas físicas marcadas como solo lectura en ambos. Cuando *cualquiera* de los dos intenta escribir, el hardware genera un page fault, el kernel hace una copia privada de esa página para el que escribe y vuelve a la ejecución. Así se evita el costo de copiar todo el espacio de direcciones, que muchas veces es seguido por un `exec` que descarta todo.

Una formulación correcta sería: “`fork` crea un nuevo descriptor en la tabla de procesos con un nuevo *PID* y le da al nuevo proceso un espacio de direcciones que es una copia lógica del padre, implementada eficientemente con copy-on-write”.

1.F — Implementación de syscalls vía *trap*

“Una llamada al sistema se implementa con una instrucción de *trap* que dispara el mecanismo de manejo de interrupciones/excepciones de la CPU.”

Respuesta

V. Exactamente. La instrucción de *trap* es la que permite cruzar la barrera de modo usuario $\rightarrow$ kernel:

- En x86-64: instrucción `syscall`.
- En ARM (AArch64): `svc #0`.
- En RISC-V: `ecall`.

Cuando la CPU encuentra la instrucción, ejecuta el mismo mecanismo que ante una interrupción/excepción: salva el estado del proceso, cambia a modo supervisor y salta a una dirección fija (la *trap table*) donde el kernel toma el control. El kernel identifica qué `syscall` pidió el proceso (el número va en un registro convenido) y la despacha a la rutina correspondiente.

El hecho de reusar el mismo *datapath* que las interrupciones/excepciones (mismo guardado de contexto, mismo mecanismo de retorno) es lo que hace que las `syscalls` sean *rápidas*: la CPU ya tiene hardware dedicado para este flujo.

2. Ejercicio 2 — Cuántos procesos crea esta cadena de forks (2 pts)

Enunciado. Dado el siguiente programa, ¿cuántos procesos se crean (además del proceso original)?

```
#include <stdio.h>
#include <unistd.h>
int main() {
    fork(); // (1)
    fork(); // (2)
    if (fork() == 0) { // (3)
        exec("myprog", NULL);
    }
    return 0;
}
```

Análisis

Cada llamada `fork()` *duplica* el proceso que la ejecuta: el invocante (padre) sigue después de la llamada y queda un nuevo proceso (hijo) que comienza a ejecutar también desde la instrucción siguiente a `fork`. Por lo tanto, después de cada `fork()` la cantidad de procesos en juego **se duplica**.

Importante: `exec("myprog", NULL)` **no** crea nuevos procesos. Reemplaza la imagen del proceso que la invoca por la del binario `myprog`, sin cambiar el PID. Para contar procesos nuevos, `exec` no suma.

Conteo paso a paso

Antes de cualquier `fork`: 1 proceso (el original, “P0”).

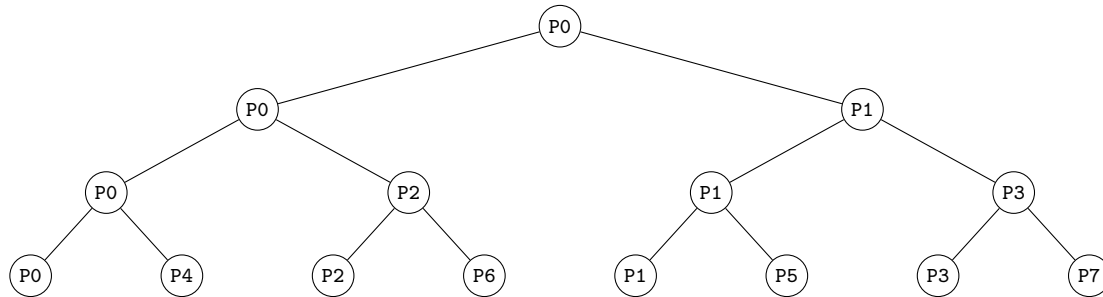
Después de `fork (1)`: cada uno de los 1 procesos se duplica $\rightarrow 1 \times 2 = 2$ procesos.

Después de `fork (2)`: $2 \times 2 = 4$ procesos.

Después de `fork (3)`: $4 \times 2 = 8$ procesos.

Notar que el `if (fork() == 0)` no impide que el `fork` se ejecute: el `fork` sí ocurre en los 4 procesos que llegan a esa línea. Los hijos creados (que reciben 0) entran al `if` y ejecutan `exec`; los padres (que reciben un PID positivo) salen del `if` y van directo al `return 0`.

Árbol de procesos



Las hojas del árbol son los 8 procesos finales (las etiquetas a la izquierda muestran “herencia”; sólo los PIDs distintos importan).

Cálculo matemático

Cantidad total de procesos = 2^k , donde k = cantidad de `fork` encadenados.

Total = $2^3 = 8$ procesos.

Procesos creados (sin contar el original) = $8 - 1 = 7$.

Respuesta

Se crean 7 **procesos nuevos** (además del original). El total en ejecución llega a 8 procesos. De esos 8:

- 4 son los “hijos” del último `fork()` (los que reciben 0) y ejecutan `exec("myprog", NULL)`, cambiando su imagen.
- Los otros 4 (incluido el original) salen del `if` sin hacer `exec` y terminan con `return 0`.

Importante: `exec` no genera procesos nuevos, así que no suma al conteo.

3. Ejercicio 3 — Comunicación con `pipe()`: ¿correcto? (2 pts)

Enunciado. Determinar si el siguiente programa es correcto para que los dos procesos (padre e hijo) se comuniquen mediante pipes y la salida sea "OK". En caso de considerarlo incorrecto, dar una versión corregida.

```

#include <stdio.h>
#include <unistd.h>
int main() {
    int p[2]; char buffer[10];
    pipe(p);
    if (fork() > 0) {
        write(p[1], "Hi", 3); // send message
        read(p[0], buffer, 10); // read response
        printf("%s\n", buffer);
    } else {
        read(p[0], buffer, 10); // read message
        write(p[1], "OK", 3); // send response
    }
    return 0;
}
  
```

Análisis: ¿Qué está mal?

El programa es **incorrecto**. El problema central es que se intenta usar **un solo pipe en ambos sentidos**, pero un pipe de UNIX es **unidireccional**: lo que se escribe en `p[1]` puede ser leído por cualquier proceso que tenga acceso al extremo de lectura `p[0]`, incluido el propio escritor.

Traza del fallo.

1. **Padre** hace `write(p[1], "Hi", 3)`. El pipe ahora contiene los bytes `H i \0`.
2. **Padre** hace `read(p[0], buffer, 10)`.
3. Si el padre lee *antes* de que el hijo lo haga (perfectamente posible, depende del scheduling), lee su **propio mensaje** "Hi" del pipe.
4. El padre imprime "Hi" en lugar del "OK" esperado.
5. Mientras tanto, el hijo se queda bloqueado en `read()` esperando un mensaje que ya fue consumido por el padre.

Aún suponiendo que el hijo gane la carrera y lea primero "Hi", quedaría un segundo problema: el hijo escribe "OK" en el mismo pipe, y entonces el padre podría leer correctamente "OK". Pero *esto depende del scheduling*: el programa no tiene **determinismo**, y bajo la mayoría de scheduling es más probable que el padre lea su propio mensaje.

Importante

Regla general: para comunicación bidireccional entre dos procesos hace falta **dos pipes**, uno por sentido. Un único pipe no alcanza porque cualquier proceso con acceso al `p[0]` puede consumir lo que el otro acaba de escribir.

Versión corregida

Usar **dos pipes**: `p` para `parent→child` y `q` para `child→parent`. Cada proceso cierra los extremos que no va a usar para forzar *EOF* cuando el otro termine.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main() {
    int p[2], q[2]; /* p: padre->hijo, q: hijo->padre */
    char buffer[10];

    pipe(p);
    pipe(q);

    if (fork() > 0) {
        /* PADRE: escribe en p, lee de q */
        close(p[0]); /* no lee de p */
        close(q[1]); /* no escribe en q */

        write(p[1], "Hi", 3); /* envia mensaje */
        close(p[1]); /* cierra para que el hijo vea EOF */

        read(q[0], buffer, 10); /* lee respuesta */
        printf("%s\n", buffer);
        close(q[0]);

        wait(NULL); /* espera al hijo (evita zombie) */
    }
```

```

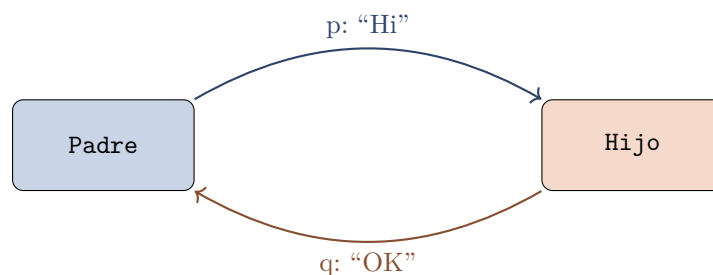
} else {
    /* HIJO: lee de p, escribe en q */
    close(p[1]); /* no escribe en p */
    close(q[0]); /* no lee de q */

    read(p[0], buffer, 10); /* lee mensaje */
    close(p[0]);

    write(q[1], "OK", 3); /* envia respuesta */
    close(q[1]);
}
return 0;
}

```

Diagrama de la comunicación corregida



Respuesta

El programa NO es correcto. Usa un único pipe en ambos sentidos, lo que provoca que el padre lea su propio mensaje “Hi” en vez de la respuesta “OK”. Además, el hijo puede quedar bloqueado indefinidamente esperando un mensaje ya consumido.

Corrección: usar *dos* pipes, uno por sentido, y cerrar los extremos que cada proceso no usa. La versión corregida está arriba.

4. Ejercicio 4 — Transferencias bancarias y *deadlock* (2 pts)

Enunciado. Un sistema bancario provee la siguiente función:

```

void transfer(Account from, Account to, double amount)
{
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);
    acquire(lock1);
    acquire(lock2);
    withdraw(from, amount); // extract money
    deposit(to, amount); // deposit money
    release(lock2);
    release(lock1);
}

```

¿Logra prevenir deadlocks? Si no, dar un escenario de dos tareas concurrentes invocando `transfer(...)` en el que pueda ocurrir deadlock. Suponer que existen al menos las cuentas `accountA` y `accountB`.

Análisis: el orden de adquisición depende de los argumentos

La función toma siempre los locks en el orden (lock1, lock2), donde lock1 corresponde a la cuenta `from` y lock2 a la cuenta `to`.

El problema: **el orden de adquisición no es global**, depende de cuál cuenta es `from` y cuál es `to` en cada invocación. Eso permite que dos invocaciones concurrentes *intenten tomar los mismos dos locks en orden inverso*, lo que es exactamente la receta clásica para un deadlock.

Escenario concreto de deadlock

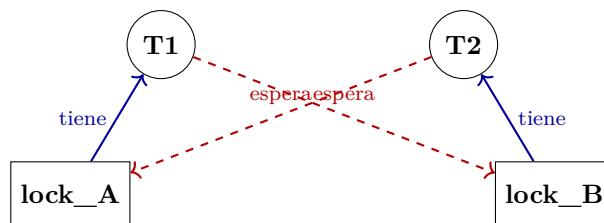
Sean dos transferencias concurrentes ejecutadas por dos threads:

<i>t</i>	Thread T1: <code>transfer(A, B, 100)</code>	Thread T2: <code>transfer(B, A, 50)</code>
<i>t</i> ₁	<code>lock1 = get_lock(A)</code>	<code>lock1 = get_lock(B)</code>
<i>t</i> ₂	<code>lock2 = get_lock(B)</code>	<code>lock2 = get_lock(A)</code>
<i>t</i> ₃	<code>acquire(lock1)</code> → obtiene <i>lock_A</i>	<code>acquire(lock1)</code> → obtiene <i>lock_B</i>
<i>t</i> ₄	<code>acquire(lock2)</code> (pide <i>lock_B</i>)	<code>acquire(lock2)</code> (pide <i>lock_A</i>)
<i>t</i> ₅	BLOQUEA : <i>lock_B</i> lo tiene T2	BLOQUEA : <i>lock_A</i> lo tiene T1

Resultado: ambos threads quedan bloqueados, esperando un lock que el otro retiene. Se cumplen las cuatro condiciones de Coffman:

- **Exclusión mutua**: cada mutex sólo puede pertenecer a un thread a la vez.
- **Hold and wait**: cada thread tiene un lock y espera otro.
- **No preemption**: el sistema no puede quitarle el mutex a un thread.
- **Espera circular**: T1 espera *lock_B* (de T2) → T2 espera *lock_A* (de T1) → T1.

Grafo de espera



Cómo prevenirlo

La técnica estándar es imponer un **orden global de adquisición** de los locks, independiente del orden de los argumentos. Una forma simple es comparar por identidad (por ejemplo, dirección de memoria o ID numérico de la cuenta) y tomar primero el menor:

```
void transfer(Account from, Account to, double amount)
{
    mutex first, second;

    /* Orden global: tomar primero el mutex de la cuenta con menor id */
    if (id(from) < id(to)) {
        first = get_lock(from);
        second = get_lock(to);
    } else {
        first = get_lock(to);
        second = get_lock(from);
    }

    acquire(first);
    acquire(second);
}
```

```

    withdraw(from, amount);
    deposit(to, amount);

    release(second);
    release(first);
}

```

Con esta corrección, *ambos* threads —no importa quién haga $A \rightarrow B$ ni $B \rightarrow A$ — tomarían *primero* `lock_A` y *después* `lock_B`. Es imposible que se forme un ciclo.

Respuesta

NO previene deadlocks. El orden de adquisición de los mutex depende de cuál cuenta sea `from` y cuál `to`; si dos hilos transfieren en sentidos contrarios ($A \rightarrow B$ y $B \rightarrow A$) simultáneamente, pueden tomar los locks en orden inverso y caer en una espera circular clásica.

Solución: imponer un *orden global* de adquisición de los locks (por ejemplo, por ID de cuenta ascendente), independiente de cuál cuenta sea `from` o `to`.

5. Ejercicio 5 — Round Robin con quantum = 2 (2 pts)

Enunciado. Asumiendo que los procesos están encolados en orden P1, P2, P3, dar un diagrama de Gantt de la planificación según un planificador Round Robin (RR) con `quantum = 2`.

Proceso	Ráfaga de uso de CPU
P1	5
P2	2
P3	3

Análisis: cómo planifica RR

Round Robin mantiene una cola FIFO de procesos **READY**. El scheduler asigna al primero un *quantum* de tiempo de CPU; al agotarse:

- Si el proceso tiene más ráfaga restante, se lo *desaloja* (*preempt*) y se lo encola al **final** de la lista.
- Si la ráfaga se completó antes o exactamente al quantum, el proceso **termina** (en este ejercicio no hay I/O).

Todos los procesos arriban en $t = 0$ y entran en el orden P1, P2, P3 a la cola **READY**.

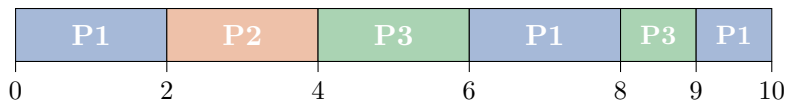
Traza paso a paso

Notación: `[P:rem]` indica el proceso P con `rem` unidades de ráfaga *remanentes*.

Intervalo	CPU	Evento al final del intervalo	Cola READY
[0, 2)	P1	quantum agotado, queda $5 - 2 = 3$	[P2:2, P3:3, P1:3]
[2, 4)	P2	P2 termina exactamente (ráfaga = 2)	[P3:3, P1:3]
[4, 6)	P3	quantum agotado, queda $3 - 2 = 1$	[P1:3, P3:1]
[6, 8)	P1	quantum agotado, queda $3 - 2 = 1$	[P3:1, P1:1]
[8, 9)	P3	ráfaga termina (queda 0), P3 EXIT	[P1:1]
[9, 10)	P1	ráfaga termina (queda 0), P1 EXIT	[]

Verificación: la CPU está ocupada de 0 a 10 sin huecos, y el total de CPU usada es $5 + 2 + 3 = 10$.
✓

Diagrama de Gantt



Métricas

Aplicando $\text{turnaround} = \text{Fin} - \text{Llegada y espera} = \text{turnaround} - \text{ráfaga}$:

Proceso	Llegada	Fin	Turnaround	Ráfaga	Espera
P1	0	10	10	5	5
P2	0	4	4	2	2
P3	0	9	9	3	6

Promedios:

Turnaround promedio = $(10 + 4 + 9) \div 3 = 23 \div 3 \approx 7.67$

Espera promedio = $(5 + 2 + 6) \div 3 = 13 \div 3 \approx 4.33$

Respuesta

Gantt: P1 | P2 | P3 | P1 | P3 | P1 con marcas de tiempo 0, 2, 4, 6, 8, 9, 10.

Tiempos de finalización: **P2 = 4, P3 = 9, P1 = 10**. P2 (la ráfaga más corta) sale primero pese a no ser la primera en encolarse; P1 (la más larga) sale al final.

Promedios: turnaround ≈ 7.67 , espera ≈ 4.33 .

Idea intuitiva

RR es **justo** (*fair*): nadie monopoliza la CPU porque los desalojos son periódicos. Pero *no* optimiza el turnaround promedio: SJF haría P2 antes (orden P2, P3, P1) y daría mejor promedio. RR sacrifica eficiencia por baja latencia de respuesta, ideal para sistemas *time-sharing* interactivos.

Resumen del recuperatorio

#	Tema	Respuesta sintética
1	V/F syscalls	A:F (exec no crea), B:F (CRT antes de main), C:F (printf es libc), D:V, E:F (COW + redacción), F:V
2	Conteo de fork	$2^3 = 8$ procesos totales $\Rightarrow$ 7 nuevos (sin contar el original)
3	Pipe bidireccional	Incorrecto: un solo pipe no separa sentidos. Corrección: dos pipes, uno por sentido
4	Deadlock en transfer	No previene; escenario T1: $A \rightarrow B$ y T2: $B \rightarrow A$. Fix: orden global de locks por ID
5	Round Robin $q=2$	Gantt: P1,P2,P3,P1,P3,P1; fin P2=4, P3=9, P1=10; turnaround promedio ≈ 7.67